三． 课程设计

简化计算机系统的设计

本实验通过实现简单的指令集的处理器和存储器连接形成简单的计算机系统，从而深入理解计算机的工作原理。

1. 处理器和内存的相互联系

程序和数据存放在内存中，处理器要和内存相互配合才能执行程序。处理器和内存之间的连接关系示于图 1 所示。

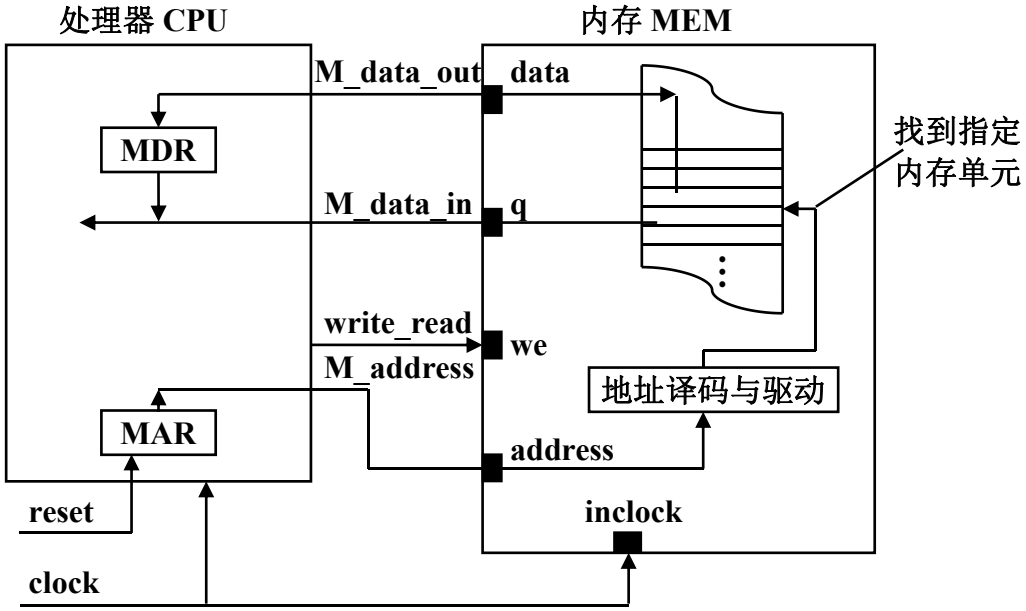


图 1 处理器和内存的相互连接

2. 指令格式

以下介绍一个简化的处理器模型（以下简称模型机）。该模型机字长 8 位，有 4 种指令格式，如下所示。

格式 1：寄存器寻址方式　　Rx←(Rx)　OP　(Ry)

15 14 13 12	11 10	9 8	7 6 5 4 3 2 1 0
OP	Rx	Ry	空白

格式 2：立即数寻址方式　　R0←I

15 14 13 12	11 10 9 8	7 6 5 4 3 2 1 0
OP	I	空白

格式 3：无操作数指令　　Rx←(Rx)　OP　(Ry)

15 14 13 12	11 10	9 8	7 6 5 4 3 2 1 0
OP	空白		空白

格式 4：直接寻址方式　　R0←→(Addr)

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OP				Addr											

指令集模型机中数据以整数的补码表示，其最高位为符号位。在对数据进行加法或减法运算时可能产生“溢出”，如果发生“溢出”，则令“溢出”标志 overflow=1。限于篇幅，本实验不讨论发生“溢出”后的处理问题。表 1 不仅定义了每一条指令所执行的操作，而且给出了该指令的助记符（表 1 第 3 列）。对此模型机而言，在没有任何软件支持的情况下，程序员面对裸机只能使用指令的二进制代码（即机器语言）编程。程序员可使用指令的助记符编程，最后再把助记符翻译成机器语言。沿此思路再向前跨一步，编写一个软件实现把指令的助记符翻译成指令的二进制代码，这就是汇编语言程序。

表 1 模型机的指令集

操作码 OP IR（15 . . 12）	指令格式	指令的助记符	操作的内容
0000	3	Idle	无操作，PC：=PC+1；（为取下一条指令准备地址）
0001	2	Load Data	R0←I；（立即数 I 送 R0）　PC：=PC+1
0010	1	Move Rx, Ry	Rx←(Ry) ;　PC := PC+l
0011	1	Add Rx, Ry	Rx←(Rx) + (Ry) ;　PC := PC+I
0100	1	Sub Rx, Ry	Rx← (Rx) - (Ry) ;　PC := PC+1
0101	1	AND Rx, Ry	Rx←(Rx) AND (Ry) ;　PC := PC+I
0110	1	OR Rx, Ry	Rx←(Rx) OR (Ry) ;　PC := PC+I
0111	1	XOR Rx, Ry	Rx←(Rx) XOR (Ry) ;　PC := PC+l
1000	1	Shr Rx	Rx←(Rx)逻辑右移 1 位；　PC:=PC+1
1001	1	Shl Rx	Rx←(Rx)逻辑左移 1 位；　PC:= PC+l
1010	1	Swap Rx, Ry	A←(Ry) ; Ry←(Rx) ; Rx←(A) ; PC := PC+1
1011	4	Jmp Addr	PC:=PC+l　取本指令的后半部 PC←Addr
1100	4	Jz Addr	PC:=PC+1 取本指令的后半部 if (R0)=0 then PC←Addr else PC:=PC+1

1101	4	Read Addr	PC:= PC+1 取本指令的后半部 R0←(addr)（从内存指定地址取数据） PC：=PC+1
1110	4	Write Addr	PC：=PC+1 取本指令的后半部 Addr←(R0)　PC：=PC+1
1111	3	Stop	无操作，PC 保持不变

3. 处理器的组成

处理器由运算器和控制器组成。控制器向运算器中各个部件发出控制信号，指挥运算器中各个部件协调工作，使数据沿指定路径在运算器中流动，从而完成整个运算。从这个意义上讲，运算器实际是各种部件适当连接的网络，数据可以在此网络中流动，因而也被称做数据通路。本节将结合模型机介绍处理器的工作原理。

3.1 寄存器组

数据通路由寄存器、多路选择器、算术逻辑运算单元(ALU)以及其他功能部件组成，下面介绍其主要功能。依照功能可将寄存器大致分为以下 7 种。

（1）通用寄存器

通用寄存器实际上是一个统一编址的寄存器堆，指令可按地址访问相应的寄存器。通用寄存器的数量由指令格式决定，例如指令格式 1 中，其访问通用寄存器的地址码长度为 2bit．所以数据通路中通用寄存器的数量应该是 2^2=4。通用寄存器用于存放原始数据和运算结果，也可以当做变址寄存器或基地址寄存器使用。

（2）累加寄存器(accumulator，Acc)或临时寄存器

一般用来存放运算的中间结果。

（3）程序计数器(program counter，PC)

程序计数器又称为指令计数器或指令指针，用于存放当前正在执行的指令地址或下面即将执行的指令地址。

（4）指令寄存器(instruction register，IR)

用于存放从内存中取出的指令，指令执行期间内容保持不变，以便控制指令的执行：

（5）内存地址寄存器(memory address register, MAR)

处理器访问内存之前，需要先将相应内存单元的地址放在 MAR 中，然后发送给内存。

（6）内存数据寄存器(memory data register，MDR)

处理器写入内存的数据需要先存放在 MDR 中，然后发送给内存；处理器从内存中读入的数据，一般先存放于此处。

(7) 状态标志寄存器(program status word register，PSWR)

为了便于控制处理器的运行，需要设一些标志位表征处理器当前的状态，例如溢出标志、进位标志、运算结果为零标志、允许中断（开中断）标志……将这些标志位合并放在一个寄存器中，就成为状态标志寄存器，又称为程序状态字寄存器。

3.2 多路选择器

多路选择器用于引导数据的流向，常用的多路器有 2 选 1、4 选 1 和 8 选 1 等。

3.3 ALU

ALU 用于执行算术运算和逻辑运算，3.7 节描述了一个简化的 ALU，由此实例可以了解 ALU 的功能。 ALU 应执行的功能由指令系统决定，当然也可以视具体情况多设几个 ALU 以分摊这些应执行的功能。

4. 模型机的结构

图 2 是模型机的结构示意图，该图对模型机的数据通路表现得比较详细，而控制器则用一个方框表示。数据通路中的主要部件包括：4 个通用寄存器（R0，R1，R2，R3），一个零时寄存器 A，1 个 ALU，1 个程序寄存器 PC，1 个指令寄存器 IR，3 个多路选择器 Multi，访问内存所需的寄存器 MAR 和 MDR。

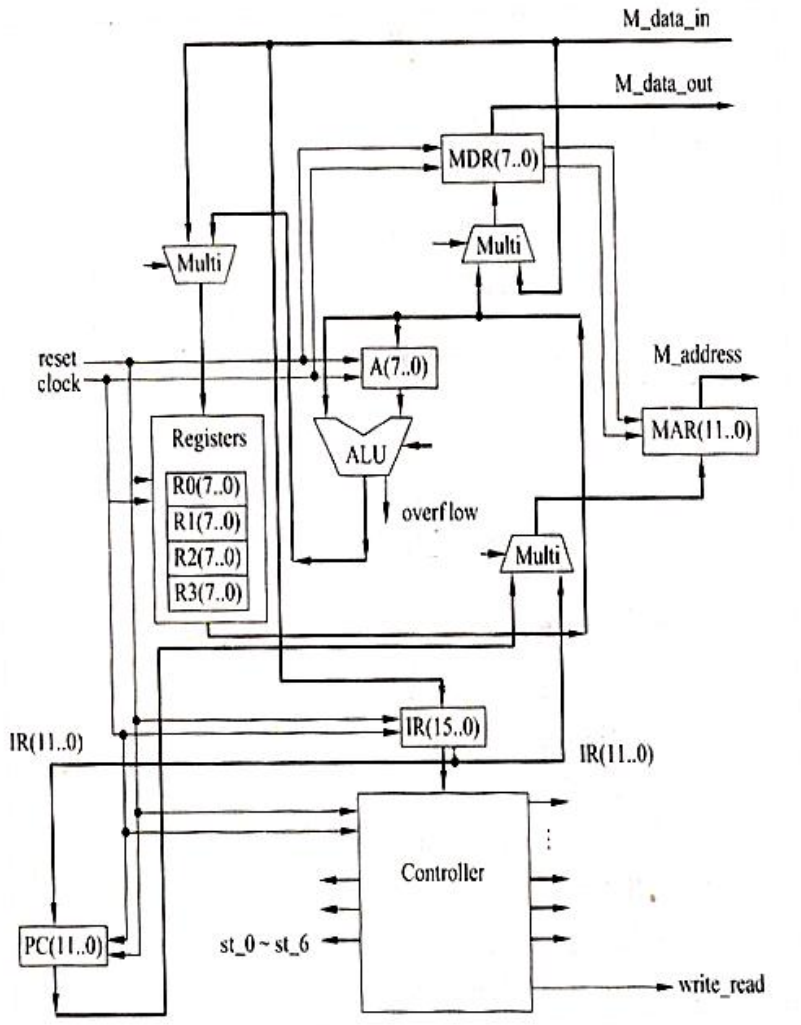


图 2 处理器结构示意图

图 2 是一个示意图，大体上表示出数据通路中的主要部件以及这些部件的互连关系，图中也画

出了这些部件所需的控制信号，这些控制信号由控制器提供，此图有助于理清思路。参照图 2，可以写出处理器行为描述的 VHDL 或 verilog HDL 源代码。

5. 处理器的设计

用 VHDL 语言或 Verilog HDL 语言写出处理器的寄存器传输级行为描述，在图 3 中用一个方框表示控制器。

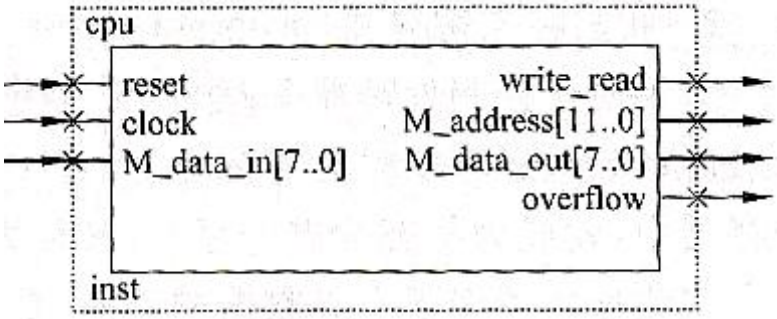


图 3 处理器的符号

6. 指令操作步骤

（1）不受时钟控制的操作

表 2 不受时钟控制的操作

信号名称	操作	是否受时钟的控制	是否受状态的控制
Reset	Reset=‘0’时，使所有寄存器清零并进入状 St_0	否	否
M_address	M_address←(MAR)	否	否
M_data_out	M_data_out← (MDR)	否	否
Write_Read	仅在状态 St_3 且指令操作码为 Write 时，Write_Read←'1';否则，Write_Read←'0'（参见表 3）	否	是

（2）其它指令的操作

- **加法指令(Add Rx, Ry)**

st_0：取指令

(l) 只和 st_0 有关而与时钟无关的操作：Write_Read←’0’，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(15 . . 8) ←M_data_in，将读出的指令加载于 IR(15 . . 8)。

② IR(7 . . 0) ← "00000000",IR 低 8 位无用，令其全部为 0。

③ PC←((PC)+1)，为后续指令准备指令地址。

(3) 时钟下降沿时执行的操作：

① MAR←(PC)，为下一条指令准备指令地址。

② A←(Ry)，Ry 由 IR(9 . . 8)指定。

③ 下一个状态转入 st_l。

st_l：执行加法操作并读下一条指令

(1) 只和 st_1 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令（读下一条指令）。

(2) 时钟上升沿时执行的操作：

Rx←(Rx)+(A)；Rx 由 IR(11 . . 10)指定。

(3) 时钟下降沿时执行的操作：下一个状态转入 st_0。

和加法指令执行过程相似的指令还有：Move, Sub, And, Or, Xor, Shr, Shl。略去其执行过程的详细描述。

- **立即数指令(Load data)**

st_0：取指令

(1) 只和 st_0 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(15 . . 8) ←M_data- in，将读出的指令加载于 IR(15 . . 8)。

② IR(7 . . 0) ←"00000000"，IR 低 8 位无用，令其为全 0。

③ PC←((PC)+1)，为后续指令准备指令地址。

(3) 时钟下降沿时执行的操作：

① MAR←(PC)，为下一条指令准备指令地址。

② A←(Ry)，Ry 由 IR(9 . . 8)指定。这是一个冗余操作，因无害而允许。

③ 下一个状态转入 st_1。

st_l：加载立即数并读下一条指令

(l) 只和 st_1 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令（读下一条指令）。

(2) 时钟上升沿时执行的操作：

① R0（3 . . 0）←IR(11 . . 8)，把立即数 data(即 IR(11 . . 8))加载于 R0 的低 4 位。

② R0(7 . . 4) ←'0000'，R0 的高 4 位全部置零。

(3) 时钟下降沿时执行的操作：下一个状态转入 st_0。

- **交换指令(Swap Rx，Ry)**

st_0：取指令

(l) 只和 st_0 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(15 . . 8) ←M_data_in，将读出的指令加载于 IR(15 . . 8)。

② IR(7 . . 0) ←"00000000"，IR 低 8 位无用，令其为全 0。

③ PC←((PC)+1)，为后续指令准备指令地址。

(3) 时钟下降沿时执行的操作：

① MAR←(PC)，为下一条指令准备指令地址。

② A←(Ry)，Ry 由 IR(9 . . 8)指定。

③ 下一个状态转入 st_1。

st_1：执行交换指令的一部分操作

(l) 只和 st_1 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令(冗余操作)。

(2) 时钟上升沿时执行的操作：Ry←(Rx)；Rx 由 IR(11 . . 10)指定，Ry 由 IR(9 . . 8)指定。

(3) 时钟下降沿时执行的操作：下一个状态转入 st_2。

st_2：执行交换指令的后续操作并读下一条指令

(1) 只和 st_2 有关而与时钟无关的操作：Write_Read='0'，向内存发出读命令（读下一条指令）。

(2) 时钟上升沿时执行的操作：Rx ← (A); Rx 由 IR(11 .. 10)指定。

(3) 时钟下降沿时执行的操作：下一个状态转入时 st_0。

- **读内存指令(Read Addr)**

st_0：取指令

(1) 只和 st_0 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(15 . . 8) ←M_data_in，将读出的指令第 1 字节加载于 IR(15 . . 8)。

② IR(7 . . 0) ←"00000000" ,IR 低 8 位无用，令其为全 0。

③ PC←((PC)+1)，为后续操作做准备。

(3) 时钟下降沿时执行的操作：

① MAR←(PC)，把本指令第 2 字节的地址加载于 MAR。

② A←(Ry), Ry 由 IR(9 . . 8)指定。这是一个冗余操作，因无害而允许。

③ 下一个状态转入 st_l。

st_l：发出读本指令的第 2 字节的命令

(l) 只和 st_l 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟下降沿时执行的操作：下一个状态转入 st_2。

st_2：将本指令的第 2 字节加载于 IR(7 . . 0)

(1) 只和 st_2 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(7 . . 0) ←M_data_in，将本指令的第 2 字节加载于 IR(7 . . 0)。至此，指令已经全部取出，存放在 IR（15 . . 0）。

② PC←((PC)+1)，为下一条指令准备指令地址。

(3) 时钟下降沿时执行的操作：

① MAR ←(IR(11 . . 0)，把取数地址加载于 MAR。

② 下一个状态转入 st_3。

st_3：向存储器发出读取数据的命令

(l) 只和 st_3 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟下降沿时执行的操作：

① MAR←(PC)，为下一条指令准备指令地址。

② 下一个状态转入 st_4。

st_4：将读出的数据加载于 R0 并发出读取下一条指令的命令

(1)只和 st_4 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

R0←M_data_in，将从存储器指定地址中读出的数据加载于 R0。

(3) 时钟下降沿时执行的操作：下一个状态转入 st_0。

- **写内存指令(Write Addr)**

St_0：取指令

(1)只和 st_0 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(15 . . 8) ←M_data_in，将读出的指令第 1 字节加载于 IR(15 . . 8)。

② IR(7 . . 0)一"00000000", IR 低 8 位无用，令其为全 0。

③ PC←((PC)+1)，为后续操作做准备。

(3) 时钟下降沿时执行的操作：

① MAR←(PC)，把本指令第 2 字节的地址加载于 MAR。

② A←(Ry), Ry 由 IR(9 . . 8)指定。这是一个冗余操作，因无害而允许。

③ 下一个状态转入 st_1。

St_1：发出读本指令的第 2 字节的命令

(l) 只和 st_1 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令，读本指令的第 2 字节。

(2) 时钟下降沿时执行的操作：下一个状态转入 st_2。

St_2：将本指令的第 2 字节加载于 IR(7 . . 0)并为写数据做准备工作

(l) 只和 st_2 有关而与时钟无关的操作：Write Read← '0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(7 . . 0) ←M_data_in，将本指令的第 2 字节加载于 IR(7 . . 0)。至此，指令已经全部取出，存放在 IR(15 . . 0)。

② PC←((PC)+1)，为下一条指令准备指令地址。

③ MDR←(R0)。

(3) 时钟下降沿时执行的操作：

① MAR←（IR(11 . . 0)），把写数地址加载于 MAR。

② 下一个状态转入 st _3。

st_3：向存储器中写数据

(1)只和 st_3 有关而与时钟无关的操作：Write_Read←'1'，向内存发出写命令。

(2) 时钟下降沿时执行的操作：

① MAR←(PC)，为下一条指令准备指令地址。

② 下一个状态转入 st_4。

st_4：读下一条指令

(1)只和 st_ 4 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令，读下一条指令。

(2) 时钟下降沿时执行的操作：下一个状态转入 st_0。

- **无条件转移指令(Jmp Addr)**

St_0：取指令

(l) 只和 st_0 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(15 . . 8) ←M_data_ in，将读出的指令第 1 字节加载于 IR(15 . . 8)。

② IR(7 . . 0) ←"00000000"，IR 低 8 位无用，令其为全 0。

③ PC←((PC)+1)，为后续操作做准备。

(3) 时钟下降沿时执行的操作：

① MAR←(PC)，把本指令第 2 字节的地址加载于 MAR。

② A←(Ry)，Ry 由 IR(9 . . 8)指定。这是一个冗余操作，因无害而允许。

③ 下一个状态转入 st_1。

st_1：发出读本指令的第 2 字节的命令

(l) 只和 st_1 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令，读本指令的第 2 字节。

(2) 时钟下降沿时执行的操作：下一个状态转入 st_2。

st_2：将本指令的第 2 字节加载于 IR(7 . . 0)并为转移做准备工作

(l) 只和 st -2 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 耐钟上升沿时执行的操作：

① IR(7 . . 0) ←M_data_ in，将本指令的第 2 字节加载于 IR(7 . . 0)。至此，指令已经全部取出，存放在 IR(15 . .0)。

② PC←((PC)+1)，这是一个冗余操作。

(3) 时钟下降沿时执行的操作：

① MAR ←(IR(11 . . 0)，把转移目的地址加载于 MAR。

② 下一个状态转入 st -3。

st_3：读下一条指令

(l) 只和 st _3 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

PC←(IR(11 . .)，把转移目的地址加载于 PC。

(3) 时钟下降沿时执行的操作：下一个状态转入 St_0。

- **条件转移指令(Jz Addr)**

St_0：取指令

(l) 只和 st_0 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(15 . . 8) ←M _data_ in，将读出的指令第 1 字节加载于 IR(15 . . 8)。

② IR(7 . . 0) ← "00000000",IR 低 8 位无用，令其为全 0。

③ PC←((PC)+1)，为后续操作做准备。

(3) 时钟下降沿时执行的操作：

① MAR←(PC)，把本指令第 2 字节的地址加载于 MAR。

② A←(Ry)，Ry 由 IR(9 . . 8)指定。这是一个冗余操作，因无害而允许。

③ 下一个状态转入 st_1。

st_1：发出读本指令的第 2 字节的命令

(l) 只和 st_1 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令，读本指令的第 2 字节。

(2) 时钟下降沿时执行的操作：下一个状态转入 st_2。

st_2：将本指令的第 2 字节加载于 IR(7 . . 0)

(1)只和 st_2 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

① IR(7 . . 0) ←M_data_in，将本指令的第 2 字节加载于 IR(7 . . 0)。至此，指令已经全部取出，存放在 IR（15 . . 0）。

② PC←((PC)+1)，形成不转移的后续指令地址。

(3) 时钟下降沿时执行的操作：

① IF(R0)=0 THEN MAR ←(IR(11 . . 0))；

ELSE MAR←(PC)； (把转移目的地址加载于 MAR)

② 下一个状态转入 st_3。

st_3：读下一条指令

(l) 只和 st_3 有关而与时钟无关的操作：Write_Read←'0'，向内存发出读命令。

(2) 时钟上升沿时执行的操作：

IF(R0)=0 THEN PC←（IR(11 . .0)），把转移目的地址加载于 PC。

(3) 时钟下降沿时执行的操作：下一个状态转入 st_0。

把每一条指令执行过程分解到相应节拍后，将其汇总填入表 4.3，就成为编写设计代码的依据。

7. 处理器每一状态下的操作及状态装换

把每一条指令的执行过程分解到相应的节拍之后，将其汇总填入表 3，就成为编写 Verilog 或 VHDL 代码的依据。

表 3 处理器每一状态下的操作及状态装换

当前状态	执行操作	状态及与访问存储器有关的操作
X	IF reset='0' THEN 所有寄存器(PC，IR，MAR，MDR…….)清 0 END IF	IF reset='0' THEN Next_state←St_0 ; END IF;

St_0	取指令：IR(15 . . 0) ← M_data_in ‖ "00000000";（符号“ll”表示连接） A←(Ry)；	Next_state ← St_1; Write_Read← ' 0' ; PC←(PC)+1 MAR ←(PC) ;

当前状态	执行操作	状态及与访问存储器有关的操作
St_1	IF OP=Load THEN R0←”0000” ‖ IR(11 . . 8);	Write_Read← '0' ; IF (OP= Stop) THEN Next_state←St_1; ELSEIF (OP=Swap OR OP=Jmp OR OP=Jz OR OP=Read OR OP=Write) THEN Next_state←st_ 2; ELSE Next_state e← st_0; END IF ;
	IF OP=Move THEN R(x) ←(A)	
	IF OP=Shr THEN Rx←(Rx)逻辑右移 1 位	
	IF OP=Shl THEN Rx←(Rx)逻辑左移 1 位	
	IF OP=Add THEN Rx←(Rx)+(A)	
	IF OP=Sub THEN Rx←(Rx)-(A)	
	IF OP=AND THEN R(x) ←(Rx) AND (A)	
	IF OP=OR THEN Rx←(Rx) OR (A)	
	IF OP=XOR THEN Rx←(Rx) XOR (A)	
	IF OP=Swap THEN Ry←(Rx)	
	IF OP=Stop THEN NULL	
	IF OP=Idle THEN NULL	
	IF OP=Jmp THEN NULL	
	IF OP=Jz THEN NULL	
	IF OP=Read THEN NULL	
	IF OP=Write THEN NULL	
St_2	IF OP = Swap THEN Rx ←(A)	Write_Read ←’0’ MDR ←(R0); IF(OP=Swap) THEN Next_state ←St_0; ELSE
	IF(OP=Jmp OR OP=READ OR OP=Write) THEN IR(7 . . 0) ←M_data_in; MAR←(IR(11 . . 0));	

	IF(OP-Jz) AND ((R0)=0) THEN IR(7 . . 0) ←M_data_in; MAR←(IR(11 . . 0)); ELSE MAR←(PC);	Next_state ←St_3; PC=(PC)+1; END IF;
St_3	IF(OP = Jmp) THEN PC=(IR(11 . . 0)); IF(OP=Jz AND (R0)=0) THEN PC=(IR(11 . . 0)) IF(OP=Read) OR (OP=Write) THEN MAR ←(PC)	IF(OP=Write) THEN Write_Read←'1'; ELSE Writa_Read←'0'; IF(OP=Jmp OR OP=Jz) THEN Next_state←St_0; ELSE Next_state←St_4;
St_4	IF(OP=Read) THEN R0 ← M_data_in;	Write_Read←'0'; Next_state←St_0;

8. 处理器与内存相连组成一台计算机

程序是指令的有序集合，程序（以及相关数据）存放在存储器模块 MEM 里。程序执行的过程，就是不断从 MEM 中取出指令并执行的过程。为了使我们的模型机工作，必须使其和一个存储器模块相连，构成一台计算机（下文称其为 computer），如图 4 所示。图中 CPU 模块与图 3 所示符号图相同，图中的存储器模块则是调用 Quartus II 的库元件 lpm_ram_dq。

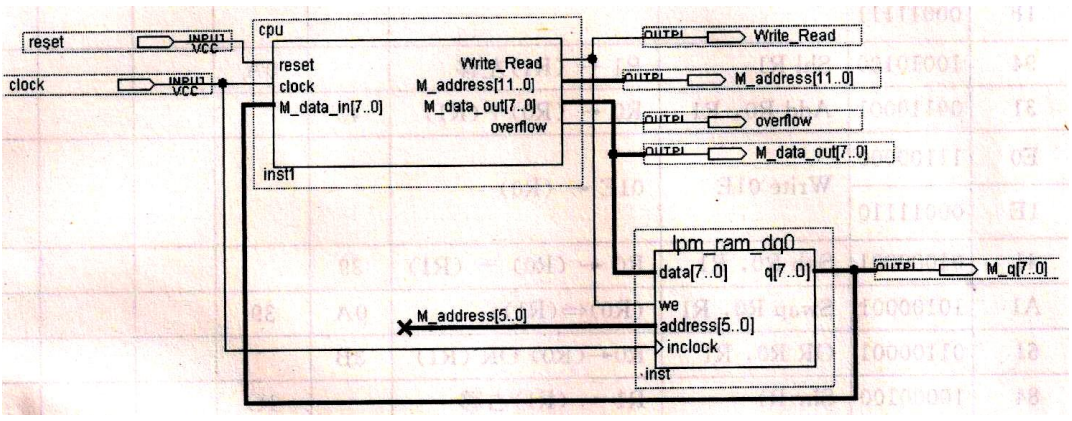


图 4 处理器和存储器相互连接

图 4 中并没有把地址总线 M_address(11..0)全部送给存储器模块，而只把其中的低 6 位 M_address(5 . . 0)送给 MEM，这是为了节省资源，以便让规模较小的器件也能容纳下本设计。事实

上，我们只在 MEM 中存放少量的调试程序，这个规模已经满足要求。此外，图中显示，CPU 的数据输入总线 M_data_in 正好是 MEM 的数据输出总线 M_q，以后把他们等同看待。为了检处理器的设计源代码是否实现预期功能，需编写一段程序（见表 4），将此调试程序预先加载到存储器模块中。

表 4　示例程序

程序地址(H)	指令代码		指令助记符	操　作	操作结果(十六进制)				
	十六进制	二进制			R0	R1	R2	R3	内存01E单元
00	00	00000000	Idle	无操作	00	00	00	00	00
01	15	00010101	Load 5	R0 ← 05	05				
02	24	00100100	Move R1,R0	R1 ← (R0)		05			
03	D0	11010000	Read 01F	R0 ← (01F)	39				
04	1F	00011111							
05	94	10010100	Shl R1	R1 ← (R1)左移		0A			
06	31	00110001	Add R0, R1	R0 ← (R0)+(R1)	43				
07	E0	11100000	Write 01E	01E ← (R0)					43
08	1E	00011110							
09	41	01000001	Sub R0, R1	R0 ← (R0) − (R1)	39				
0A	A1	10100001	Swap R0, R1	(R0)⇔(R1);	0A	39			
0B	61	01100001	OR R0, R1	R0←(R0) OR (R1)	3B				
0C	84	10000100	Shr R1	R1 ← (R1)右移		1C			
0D	51	01010001	AND R0, R1	R0←(R0) AND (R1)	18				
0E	28	00101000	Move R2, R0	R2 ← (R0)			18		
0F	2D	00101101	Move R3, R1	R3 ← (R1)				1C	
10	7B	01111011	XOR R2, R3	R2 ← (R2) XOR (R3)			04		
11	D0	11010000	Read 01E	R0 ← (01E)	43				
12	1E	00011110							
13	C0	11000000	Jz 019	若(R0)＝ 0 转移至019，否则执行下一条					
14	19	00011001							
15	D0	11010000	Read 01D	R0 ← (01D)	00				
16	1D	00011101							
17	B0	10110000	JMP 013	无条件转移至 013					
18	13	00010011							
19	F0	11110000	Stop	停止执行					
1A	00	00000000	(数据)						
1B	00	00000000	(数据)						
1C	00	00000000	(数据)						
1D	00	00000000	(数据)						
1E	00	00000000	(数据)						
1F	39	00111001	(数据)						

于是图 4 所示的 Computer 就可以开始工作了。作为实例，我们把在 Quartus II 环境下的操作步骤陈述如下：

第一步：创建一个新项目 cpu. qpf，其顶层文件为 cpu. vhd。

第二步：对上述项目 cpu. qpf 进行编译。

第三步：编译成功后为其创建一个符号图，即图 1 所示符号图。

第四步：创建一个新项目 computer. qpf，在例化存储器模块库元件 lpm_ram_dq 时，要为其指定一个初始化存储器内容的文件（例如名字为 cpu_mem. mif)，将表 4 所示调试程序的十六进制代码（也可以用二进制代码）填写在文件中。